ĐỀ CƯƠNG ĐỀ TÀI

POINT-IN-TIME CORRECT FEATURE STORE

Tên tiếng Anh: Point-in-Time Correct Feature Store
Lĩnh vực: Data Engineering, Feature Platform, Machine Learning
Thời gian thực hiện dự kiến: 5 tuần
Bộ dữ liệu kiểm chứng: IEEE-CIS Fraud Detection
Use case minh họa: Tạo historical features cho mô hình phát hiện gian lận giao dịch

---

1. Tóm tắt đề tài

Đề tài xây dựng một Feature Store thu nhỏ có khả năng tạo và cung cấp feature lịch sử đúng tại thời điểm dự đoán (Point-in-Time Correct). Feature của một observation tại thời điểm `T` chỉ được phép sử dụng dữ liệu đã xảy ra trước `T`; hệ thống không được nhìn thấy sự kiện tương lai, nhãn tương lai hoặc các thống kê được tính trên toàn bộ dữ liệu.

IEEE-CIS Fraud Detection được chọn làm bộ dữ liệu kiểm chứng vì có dữ liệu giao dịch theo thời gian, nhiều entity candidates và nhãn để đánh giá khả năng sử dụng feature. Phát hiện gian lận chỉ là **use case minh họa cho platform**, không phải giới hạn của chủ đề. Kiến trúc và các nguyên tắc Point-in-Time có thể áp dụng cho recommendation, churn prediction, credit scoring, demand forecasting hoặc các bài toán ML có dữ liệu biến đổi theo thời gian.

Sản phẩm không tập trung tối đa điểm thi Kaggle. Mục đích chính là giải quyết rủi ro sai lệch thời gian khi tạo feature cho Machine Learning: ngăn dữ liệu tương lai đi vào feature quá khứ, quản lý feature nhất quán và có version, tái tạo được training dataset, hỗ trợ backfill an toàn, đồng thời duy trì tính đúng đắn khi pipeline được tối ưu hoặc chạy lại.

2. Bối cảnh và vấn đề cần giải quyết

Trong nhiều hệ thống ML, mô hình sử dụng dữ liệu lịch sử để chấm điểm một observation mới. Nếu pipeline offline vô tình dùng các sự kiện xảy ra sau thời điểm cần dự đoán, kết quả đánh giá mô hình sẽ cao không thực tế. Hiện tượng này gọi là future data leakage.

Với use case IEEE-CIS, một ví dụ là số giao dịch của cùng thẻ trong 24 giờ, tổng giá trị giao dịch trong 7 ngày hoặc thời gian kể từ giao dịch gần nhất. Các feature này là phương tiện để kiểm thử Feature Store, không phải định nghĩa của platform.

Ví dụ, khi cần chấm điểm giao dịch lúc 10:00, feature `txn_count_24h` chỉ được đếm các giao dịch trước 10:00. Nếu giao dịch lúc 11:00 được đưa vào feature của 10:00, dataset huấn luyện đã vi phạm Point-in-Time correctness.

Đề tài giải quyết các câu hỏi:

1. Làm thế nào tạo historical features mà không dùng dữ liệu tương lai?
2. Làm thế nào bảo đảm dữ liệu mới được thêm vào không làm thay đổi feature của quá khứ?
3. Làm thế nào quản lý version, lineage và khả năng tái tạo feature?
4. Làm thế nào tạo training dataset và offline feature table nhất quán?
5. Làm thế nào kiểm thử, backfill và tối ưu pipeline mà không làm thay đổi kết quả logic?

3. Mục tiêu

3.1. Mục tiêu tổng quát

Xây dựng một Feature Store chạy batch có khả năng định nghĩa, tạo, lưu trữ và tái sử dụng historical features đúng tại thời điểm cần dự đoán. Hệ thống phải tạo được training dataset phản ánh đúng lượng thông tin thực sự có sẵn tại mỗi thời điểm, nhờ đó giảm future leakage và làm cho kết quả đánh giá offline đáng tin cậy hơn. IEEE-CIS Fraud Detection được dùng làm use case thực nghiệm để triển khai và đánh giá giải pháp trên dữ liệu thực.

3.2. Mục tiêu cụ thể

- Kiểm kê và kiểm tra chất lượng bộ dữ liệu IEEE-CIS.
- Chuyển raw CSV thành Bronze và canonical Silver datasets.
- Tạo `event_time` từ `TransactionDT` bằng một synthetic epoch cố định.
- Xây dựng primary và secondary surrogate entity keys.
- Định nghĩa feature bằng cấu hình có version.
- Tính rolling features với cutoff `history_time < prediction_time`.
- Materialize offline feature table và hỗ trợ backfill idempotent.
- Chia train/validation/test theo thời gian, không random split làm đánh giá chính.
- Viết automated tests để phát hiện future leakage và sai window boundary.
- Huấn luyện một baseline model trên use case fraud detection để đánh giá khả năng sử dụng của dữ liệu đầu ra cho ML.
- Đo runtime và lợi ích của partition pruning hoặc các tối ưu PIT pipeline.

4. Nguồn dữ liệu

4.1. Nguồn chính

Đề tài sử dụng bộ dữ liệu **IEEE-CIS Fraud Detection** trên Kaggle làm dữ liệu thực nghiệm và kiểm chứng platform:

https://www.kaggle.com/competitions/ieee-fraud-detection/data

Dữ liệu do IEEE Computational Intelligence Society phối hợp cùng Vesta Corporation cung cấp cho mục đích nghiên cứu phát hiện gian lận. Nhiều trường đã được ẩn danh hoặc che giấu ý nghĩa để bảo vệ

thông tin nghiệp vụ và quyền riêng tư. Việc chọn dataset này không làm kiến trúc Feature Store phụ thuộc riêng vào ngành Fintech.

4.2. Các file đầu vào

File	Vai trò	Quy mô quan sát
`train_transaction.csv`	Giao dịch train và nhãn `isFraud`	590.540 dòng, 394 cột
`train_identity.csv`	Identity/device cho một phần giao dịch train	144.233 dòng, 41 cột
`test_transaction.csv`	Giao dịch Kaggle test, không có nhãn công khai	506.691 dòng, 393 cột
`test_identity.csv`	Identity/device cho một phần giao dịch test	141.907 dòng, 41 cột
`sample_submission.csv`	Định dạng submission Kaggle, chỉ dùng tùy chọn	506.691 dòng

`TransactionID` là khóa nối transaction và identity. Do identity chỉ có cho một phần giao dịch, pipeline phải dùng left join để không làm mất transaction.

4.3. Nhóm trường dữ liệu quan trọng

- `TransactionID`: định danh giao dịch, được kỳ vọng không null và duy nhất.
- `TransactionDT`: số giây lệch từ một mốc bí mật, không phải timestamp thật.
- `TransactionAmt`: giá trị thanh toán bằng USD.
- `ProductCD`: mã sản phẩm hoặc dịch vụ.
- `card1`–`card6`: thông tin thẻ đã được mã hóa.
- `addr1`, `addr2`: billing region và billing country đã được mã hóa.
- `P_emaildomain`, `R_emaildomain`: email domain của purchaser và recipient.
- `C1`–`C14`, `D1`–`D15`, `M1`–`M9`, `V1`–`V339`: các nhóm biến đếm, thời gian, match và feature do Vesta tạo.
- `id_01`–`id_38`, `DeviceType`, `DeviceInfo`: thông tin identity, network, browser, OS hoặc device đã được ẩn nghĩa một phần.
- `isFraud`: nhãn fraud của training data.

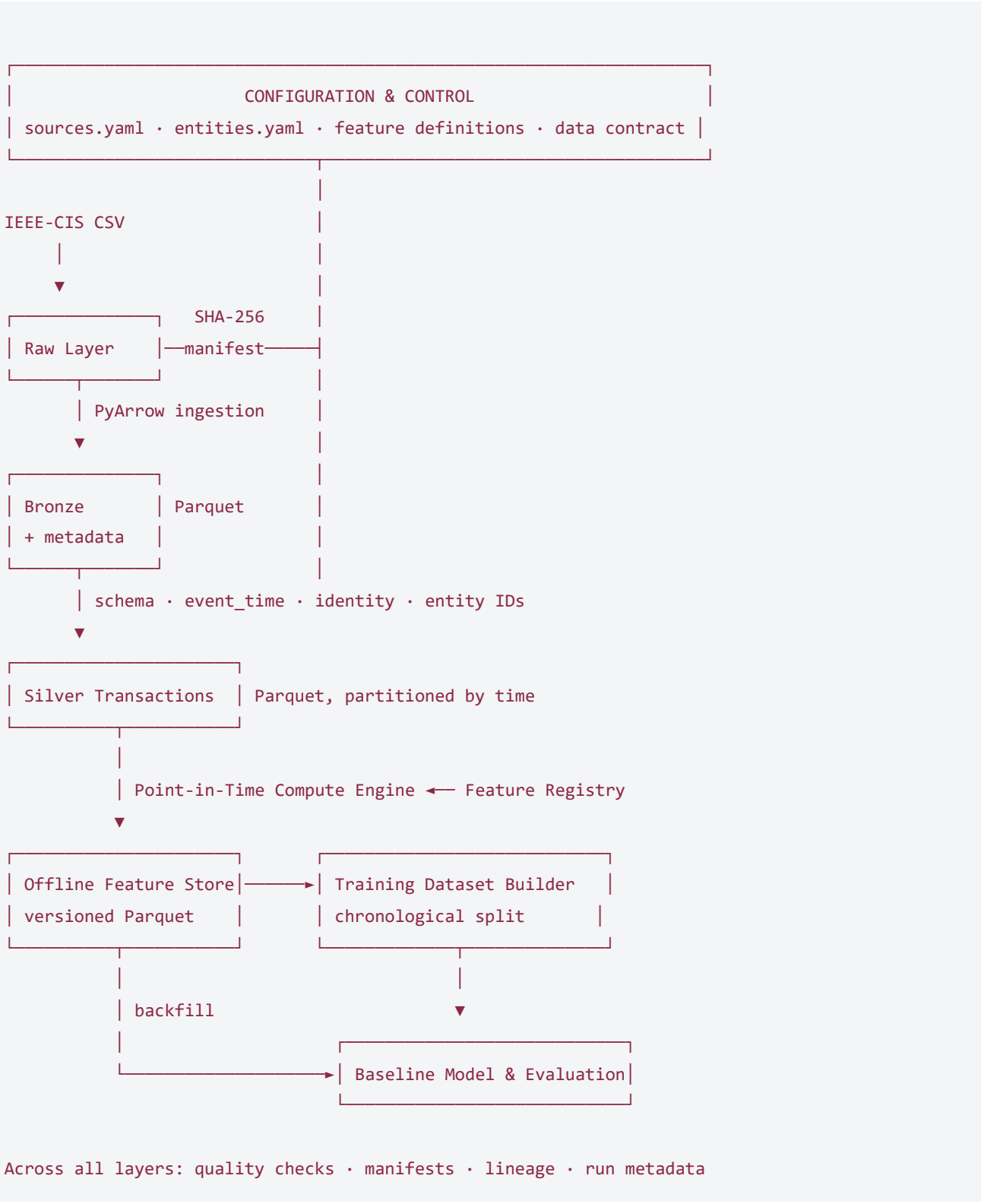
4.4. Đặc điểm và giới hạn

- Train có 20.663 giao dịch fraud, tương đương khoảng 3,499%; dữ liệu mất cân bằng mạnh.
- `TransactionDT` chỉ bảo toàn thứ tự và khoảng cách thời gian. Project dùng synthetic epoch `2017-01-01T00:00:00Z` để tạo timestamp kỹ thuật; không diễn giải đây là ngày giao dịch thật.
- Dataset không cung cấp customer/account ID chính thức.
- Nhiều trường bị masked, vì vậy project không suy luận danh tính khách hàng thật.
- Kaggle test không có nhãn công khai; đánh giá nội bộ dùng chronological split từ train.
- Raw dataset và Kaggle credential không được commit lên GitHub.

5. Kiến trúc hệ thống và thiết kế dữ liệu

5.1. Kiến trúc tổng thể

Kiến trúc MVP được thiết kế theo hướng batch-first, gồm các tầng tiếp nhận dữ liệu, chuẩn hóa, tính Point-in-Time features, lưu trữ offline và tạo training dataset. Các cấu hình, manifest và quality reports được lưu riêng để bảo đảm versioning, lineage và reproducibility.



Các thành phần chính:

Thành phần	Vai trò
Configuration	Khai báo source, synthetic epoch, entity và feature version; pipeline không hard-code logic.
Raw Layer	Lưu CSV nguyên gốc, coi là immutable và xác minh bằng SHA-256.
Bronze Layer	Chuyển source sang Parquet, giữ giá trị nguồn và bổ sung ingestion metadata.
Silver Layer	Tạo canonical transaction schema, `event_time`, identity flags và entity IDs.
Feature Registry	Định nghĩa entity, window, aggregation, cutoff policy và version của feature.
PIT Compute Engine	Tính rolling historical features theo đúng cutoff và entity.
Offline Feature Store	Lưu feature có version bằng Parquet, phục vụ backfill và training.
Training Dataset Builder	Tạo dataset theo chronological split và lưu manifest tái lập.
Quality & Metadata	Kiểm tra row count, uniqueness, PIT correctness, checksum và trạng thái từng run.

5.2. Các tầng dữ liệu

```
Kaggle CSV
    ↓
Raw: file tải nguyên gốc, immutable, kiểm tra SHA-256
    ↓
Bronze: dữ liệu gần nguyên bản ở Parquet + ingestion metadata
    ↓
Silver: schema chuẩn + event_time + identity + surrogate entities
    ↓
Point-in-Time feature computation
    ↓
Offline Feature Store
    ↓
Training Dataset và Baseline Model
```

5.3. Thời gian sự kiện

```
event_time = 2017-01-01T00:00:00Z + TransactionDT seconds
```

Synthetic epoch chỉ là mốc kỹ thuật. Phép biến đổi giữ nguyên thứ tự và khoảng cách giữa các giao dịch, nhờ đó có thể tính window 1 giờ, 24 giờ, 7 ngày hoặc 30 ngày.

5.4. Surrogate entities

Do không có customer ID, MVP sử dụng hai entity view:

```
Primary:   card_address_entity_id = SHA-256(v1, card1, addr1)
Secondary: card_entity_id         = SHA-256(v1, card1)
```

Primary cụ thể hơn nhưng không tồn tại khi thiếu `addr1`. Secondary có coverage cao hơn nhưng gom lịch sử rộng hơn. Mô hình nhận cả hai feature sets với tên riêng:

- `card_addr_*`: lịch sử của cặp card-address.
- `card_*`: lịch sử rộng hơn của card.

Hai view không được trộn vào cùng một feature column.

6. Phương pháp thực hiện

Bước 1 — Dataset audit

Đọc metadata và schema, kiểm tra row count, null rate, cardinality, `TransactionID`, `isFraud`, identity coverage, phạm vi `TransactionDT`, checksum và train/test compatibility.

Bước 2 — Bronze/Silver foundation

Ingest CSV sang Parquet, lưu source checksum và run metadata. Chuẩn hóa kiểu dữ liệu, kiểm tra duplicate, tạo `event_time`, left join identity, tạo `has_identity`, gắn hai entity IDs và routing metadata.

Bước 3 — Feature Registry

Định nghĩa feature bằng YAML có các trường: tên, version, entity, source, timestamp, aggregation, window, cutoff policy, owner và description.

Bước 4 — Point-in-Time computation

Mỗi transaction là một observation tại `prediction_time = event_time`. Historical record chỉ hợp lệ khi:

```
history.entity_id = observation.entity_id
history.event_time < observation.prediction_time
history.event_time >= observation.prediction_time - window
```

Bước 5 — Offline store và training dataset

Materialize feature table có version ra Parquet, partition theo thời gian. Chia training/validation/internal-test theo chronological order và fit preprocessing chỉ trên training period.

Bước 6 — Kiểm thử và đánh giá

Tạo golden dataset có expected results tính bằng tay. Kiểm tra no-history, null entity, same timestamp, window boundary, entity isolation và future event. Huấn luyện baseline model và đánh giá bằng ROC-AUC, PR-AUC, Recall tại FPR cố định và Precision@K.

7. Feature dự kiến

Mỗi feature lịch sử được tính cho cả primary và secondary entity khi khả dụng:

- Số giao dịch trong 1 giờ, 6 giờ, 24 giờ và 7 ngày.
- Tổng amount trong 1 giờ, 24 giờ và 7 ngày.
- Average/max amount trong 24 giờ hoặc 7 ngày.
- Tỷ lệ amount hiện tại so với average lịch sử.
- Số giây từ giao dịch trước.
- Số giây từ lần đầu entity xuất hiện.
- Số ProductCD khác nhau trong lịch sử.
- Số email domain hoặc device khác nhau nếu dữ liệu đủ tin cậy.
- `has_identity`, `has_card_address_entity` và `entity_resolution_level`.

8. Output mong muốn chi tiết

8.1. Source inventory và quality reports

- Dataset profile dạng JSON cho máy đọc.
- Báo cáo audit dạng Markdown/PDF.
- SHA-256 manifest cho raw files.
- Data dictionary và data contract.

8.2. Bronze datasets

- Bốn bảng Bronze Parquet tương ứng với bốn CSV chính.
- Giữ nguyên giá trị nguồn và bổ sung `source_file`, `source_checksum`, `ingested_at`, `ingestion_run_id`.
- Có row-count reconciliation giữa Raw và Bronze.

8.3. Canonical Silver dataset

Một bảng `silver_transactions` chứa:

- Các trường transaction và identity đã chuẩn hóa.
- `event_time` và partition thời gian.
- `has_identity`.
- `card_address_entity_id`, `card_entity_id`.
- `has_card_address_entity`, `entity_resolution_level` và entity versions.

- Quality report chứng minh không mất hoặc nhân bản transaction sau left join.

8.4. Feature Registry

- Feature definitions YAML/JSON có version.
- Feature catalog giải thích ý nghĩa và cutoff của từng feature.
- Validator tự động từ chối definition thiếu entity, source, timestamp, window hoặc version.

8.5. Point-in-Time feature table

- Offline feature table dạng Parquet, partition theo observation time.
- Chứa cả `card_addr_*` và `card_*` feature sets.
- Có feature version, source checksum, computation run ID và cutoff metadata.
- Số output observations bằng số input observations.

8.6. Automated correctness tests

- Golden tests có expected values tính bằng tay.
- Test future event không ảnh hưởng past feature.
- Test window boundary, same timestamp, null/no-history và entity isolation.
- Test chạy lại/backfill không tạo duplicate.

8.7. Reproducible training dataset

- Train/validation/internal-test chia theo thời gian.
- Manifest ghi source version, feature versions, cutoff range, row count và checksum.
- Preprocessing chỉ fit trên training period.

8.8. Baseline model và evaluation report

- Raw-feature baseline và PIT-feature baseline.
- Báo cáo ROC-AUC, PR-AUC, Recall@FPR và Precision@K.
- So sánh kết quả toàn bộ dữ liệu và theo `entity_resolution_level`.
- Model chỉ dùng để đánh giá feature usability, không đặt mục tiêu tối ưu Kaggle leaderboard.

8.9. Performance benchmark và demo

- Runtime, files/partitions/bytes scanned trước và sau tối ưu.
- Chứng minh partition pruning hoặc semi-join reduction không thay đổi feature values.
- Demo trực quan một giao dịch tại cutoff `T`, trong đó future event bị loại.
- README, architecture, runbook, báo cáo cuối và slide trình bày.

9. Tiêu chí nghiệm thu

1. Không có history record vi phạm cutoff của feature.
2. Thêm giao dịch tương lai không làm thay đổi feature đã tạo cho quá khứ.

3. Số Silver và feature observations được reconciliation với nguồn.
4. Pipeline chạy lại cho kết quả nghiệp vụ giống nhau và backfill không duplicate.
5. Golden PIT tests pass 100%.
6. Training dataset không temporal overlap giữa train, validation và internal test.
7. Feature definitions, source checksum và dataset manifest cho phép tái tạo kết quả.
8. Có ít nhất 8 historical features, baseline model và báo cáo metric.
9. Có benchmark hiệu năng và demo tái lập được từ README.

10. Phạm vi và giới hạn

Phạm vi bắt buộc

- Batch ingestion và Bronze/Silver layers.
- Surrogate entities và versioned Feature Registry.
- Batch PIT computation và Offline Feature Store.
- Chronological training dataset, leakage tests và baseline model.
- Báo cáo correctness, quality và performance.

Không thuộc phạm vi bắt buộc

- Streaming production chịu tải thực tế.
- Kubernetes hoặc cloud production deployment.
- Customer identity resolution chính xác.
- Deep learning/ensemble phức tạp.
- Tối ưu Kaggle leaderboard.
- Online Redis/FastAPI nếu tiến độ 5 tuần không cho phép.

11. Kế hoạch 5 tuần

Tuần	Trọng tâm	Sản phẩm đầu ra
1	Chốt đề tài, audit dữ liệu và entity analysis	Proposal, profile, dictionary, entity decision
2	Bronze/Silver, registry và PIT skeleton	Canonical data, feature definitions, golden dataset
3	Hoàn thiện MVP trên dữ liệu thật	PIT engine, offline table, training dataset, demo
4	Leakage audit, model, tối ưu và edge cases	Metrics, correctness report, benchmark
5	Hoàn thiện tài liệu và trình bày	Báo cáo, README, slide và demo ổn định

12. Giá trị dự kiến

Kết quả của đề tài giúp quá trình xây dựng dataset cho Machine Learning phản ánh đúng bối cảnh dự đoán thực tế. Historical features được tạo theo một cutoff rõ ràng, có version và có thể truy vết về nguồn dữ liệu, nhờ đó hạn chế future leakage và tránh kết quả đánh giá offline cao giả tạo.

Feature Store cũng tạo một quy trình thống nhất để định nghĩa, materialize, backfill và tái sử dụng feature giữa nhiều lần thử nghiệm. Khi dữ liệu mới xuất hiện hoặc pipeline được tối ưu, hệ thống có các kiểm thử để bảo đảm feature quá khứ không bị thay đổi sai và kết quả logic vẫn nhất quán. IEEE-CIS cung cấp tình huống thực nghiệm cụ thể; kết quả chính là phương pháp và platform Point-in-Time Correct có thể áp dụng cho những bài toán ML khác có dữ liệu thay đổi theo thời gian.

13. Tài liệu tham khảo

1. Kaggle, IEEE-CIS Fraud Detection: https://www.kaggle.com/competitions/ieee-fraud-detection/overview
2. Kaggle, IEEE-CIS Fraud Detection Data: https://www.kaggle.com/competitions/ieee-fraud-detection/data
3. Liu et al., *Optimizing Data Pipelines for Machine Learning in Feature Stores*, PVLDB 2023: https://www.vldb.org/pvldb/vol16/p4230-camacho-rodriguez.pdf